

Unique paper code: 6203450011

Name of Paper: Design and Analysis of Algorithms

Name of the Course: B.Voc Software Development

Semester: 5

Duration: 3 Hours

Maximum Marks: 90

Instructions for Candidates:

- Write your Roll No. on the top immediately on receipt of question paper.
- The paper has two sections. Section A is compulsory. Each question is of 5 marks.
- Attempt any four questions from Section B. Each question is of 15 marks.

Section A

- What are Omega, Theta, Big-oh notations? Give an example for each notation. 5
 - Explain the working of Bubble Sort for the given list [9, 8, 3, 6, 5, 2, 10] 5
 - Describe the concept of Divide and Conquer with respect to Merge Sort by using a suitable example. 5
 - Differentiate between NP-hard and NP-complete problems with examples. 5
 - What is the difference between Greedy and Dynamic Programming approaches? 5
 - What is Linear Time Sorting? Sort the given list [402, 308, 334, 555, 777, 112, 888] using Radix Sort. 5

Section B

- Explain any one method to test Check whether the following graph is bipartite: 8
Vertices = {A, B, C, D, E}, Edges = {(A,B), (A,C), (B,D), (C,D), (D,E)}.
 - Define a Directed Acyclic Graph (DAG) and explain how to find its Topological Ordering with an example. 7
- Explain the working of Quick Sort with the array [8, 3, 1, 7, 0, 10, 2] using pivot = 8. Show all recursive steps. 8
P P P R R
 - Derive and explain the average-case and worst-case time complexities of Quick Sort. 7
- Apply Kruskal's algorithm to find the MST of this weighted graph: 8

Edge	Weight
A-B	4
A-C	3
B-C	1

B-D	2
C-D	4
D-E	2

Show detailed steps and final MST.

- b) Using the greedy algorithm approach, Solve a Fractional Knapsack problem with 3 items (values = {40, 120, 80}, weights = {15, 25, 30}, Capacity = 60. 7
- 5 a) Explain the Dynamic Programming approach for solving the 0/1 Knapsack problem with a numerical example: 8
Items = {(weight, value)} = {(3, 13), (4, 20), (5, 30)}, Capacity = 8.
- b) Explain the Divide and Conquer strategy with a step-by-step illustration of the Maximum Subarray Problem. 7
- 6 a) Perform Bubble Sort on [8, 4, 7, 3, 9], showing each pass. Analyse its best, average, and worst-case time complexity. 8
- b) Write a python script to implement DFS algorithm for graphs and discuss its time complexity. 7
- 7 a) Define NP-hard problems and explain why Subset Sum Problem is NP-hard. 8
- b) Explain the difference between Decision and Optimization problems with suitable examples. 7